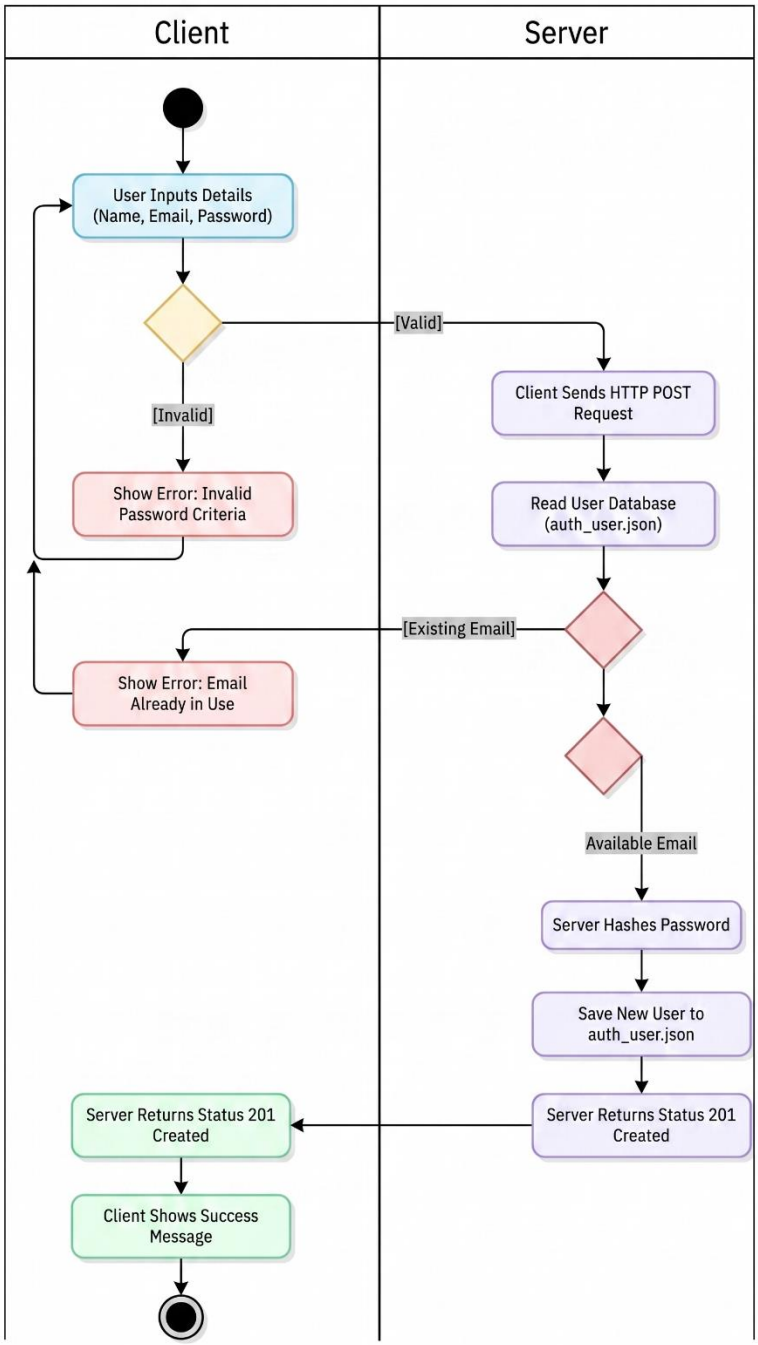
Tonight Work

1. Write the “Contract” Table.

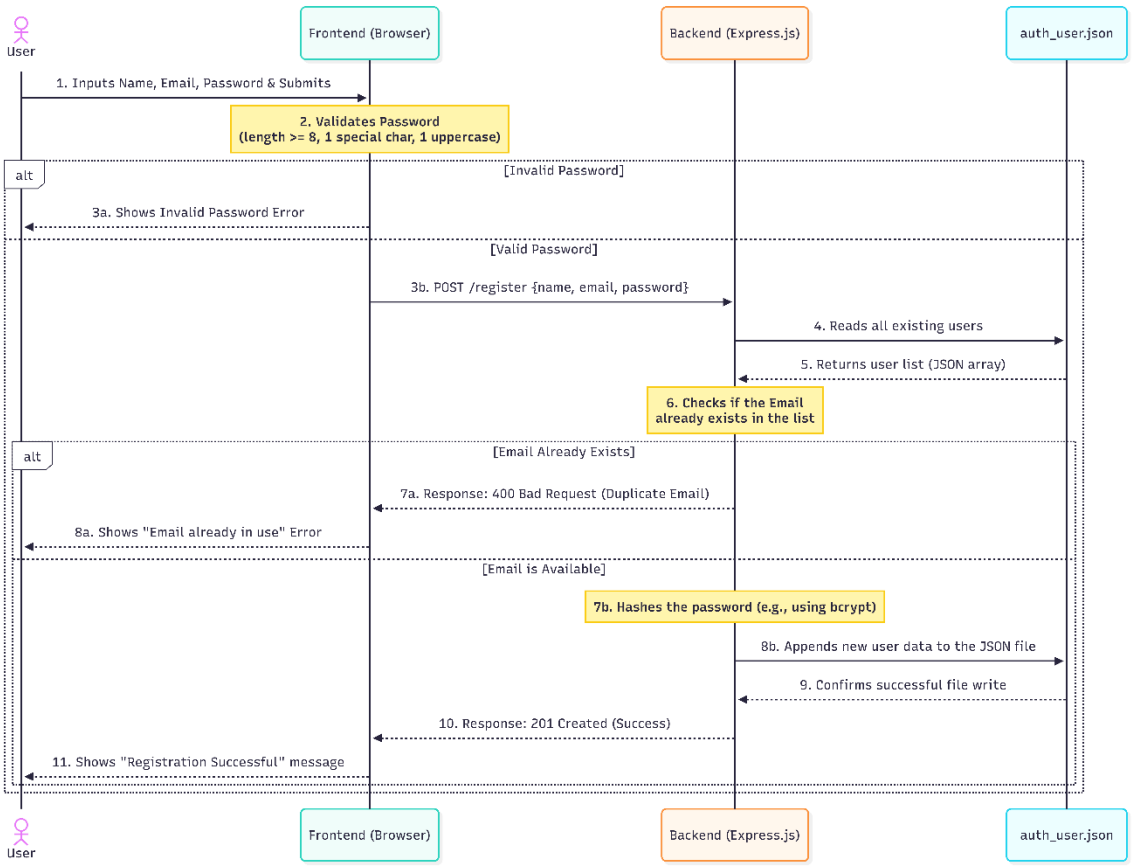
Component	Request (Order)	Response (Delivery)
Method	POST	-
Endpoint (URL)	/api/auth/register	-
Request Header	Content-Type: application/json	-
Response Header	-	Content-Type: application/json
Status Code	-	201 Created, 400 Bad Request, 409 Conflict, 500 Server Error
Body (Request)	{ "firstName": "Somchai", "email": "somchai@email.com", "password": "Pass123!" }	-
Body (Response)	-	{ "success": true, "message": "Registration successful!", "user": { "id": 11, "firstName": "Somchai", "email": "somchai@email.com", "registrationDate": "2025-05-04T..." } }
Password Rules	Min **8 characters** · At least **1 uppercase** (A–Z) · At least **1 special character** (! @ # $ % ^ & *)	

2. Write an Activity Diagram and a Sequence Diagram for user registration.

Activity Diagram:



Sequence Diagram:



3. Write GenAI Prompt from the "Contract" table + Diagrams

Prompt:

Act as a Security Engineer building a Node.js/Express backend.

Context:

- Project: FreshmartOrganic e-commerce site
- User data is stored in /data/auth_user.json as a JSON array
- Follow the same class-based service pattern as productService.js - Flow follows this Sequence Diagram:

1. Client sends POST, 2. Server validates input, 3. Server checks DB for existing email, 4. Server hashes password with bcrypt, 5. Server saves to JSON, 6. Server returns 201.

Task:

Write a POST route for /api/auth/register that:

1. Accepts JSON body: { firstName, email, password }
2. Returns 400 if any field is missing or email format is invalid
3. Returns 400 if password fails any rule (see Constraints)
4. Returns 409 if the email already exists in auth_user.json
5. Hashes the password securely using the bcrypt library (salt rounds: 10)
6. Appends the new user to auth_user.json with auto-increment ID and registrationDate
7. Returns 201 with { success: true, message, user } — never return the password hash

Constraints (from Contract Table):

- Password: min 8 chars, at least 1 uppercase (A-Z), at least 1 special char (!@#$%^&*)
- Validate on the SERVER side even if frontend already checked
- Separate route file (routes/auth.js) and service file (services/register.js)
- Add JSDoc comments on every method

Format:

- Output two files: routes/auth.js and services/register.js
- Use CommonJS (require/module.exports)
- Add inline comments explaining each security decision

Act as a Frontend Developer.

Context:

- Existing file: user-register.html (FreshmartOrganic project)
- Backend API: POST /api/auth/register returns 201/400/409 with JSON

Task:

Add a <script> block to user-register.html that:

1. Validates password LIVE as the user types:
- Shows a checklist with 3 items (8+ chars, uppercase, special char)
- Each item turns green ✓ when the rule passes, stays red ✗ if not
2. On form submit (prevent default):
- Checks all rules before calling the API
- Calls POST /api/auth/register with fetch()
- Shows a success alert and redirects to user-login.html after 2 seconds
- Shows an error alert with the message from the API on failure
3. Disables the submit button while the API call is in progress

Constraints:

- Use vanilla JS only (no jQuery for the register logic)
- Must work with the existing Bootstrap CSS classes already in the file
- Add comments explaining each step

At the top of services/register.js, please include the following Logic Flow as a block comment to explain how it works:

1. Trigger: User submits the registration form on user-register.html

2. Request: Browser sends POST /api/auth/register with { firstName, email, password }

3. Processing (Gatekeeper):

- Validate input format (Server-side check)

- Check if email already exists in auth_user.json (Prevent duplicates)

- Hash password using bcrypt (One-way encryption for security)

4. Data Persistance: Append new user object to auth_user.json

5. Response: Server returns { success: true, message: "..." } with status 201 (Never return the password hash)